

PROFESSIONAL UNIVERSAL MERN STACK PROJECT PROMPT

Create a complete professional Full Stack MERN Application for a National Practical Exam style project.

The project must contain a professional frontend, scalable backend architecture, responsive UI/UX, authentication system, CRUD operations, reporting system, and API integration.

The system should look modern, responsive, attractive, and professional like a real-world production application.

PROJECT INFORMATION

Project Name

[ADD YOUR PROJECT NAME HERE]

Project Description

[ADD YOUR PROJECT DESCRIPTION HERE]

Main Entities

[ADD YOUR ENTITIES HERE]

Example:

- Students
- Teachers
- Courses
- Payments

OR

- Cars
- Parking Slots
- Parking Records
- Payments

TECHNOLOGY STACK

Frontend

Use:

- React.js
- React Router DOM
- Axios
- Tailwind CSS
- React Icons
- Context API

Backend

Use:

- Node.js
- Express.js

Database

Use one:

- MongoDB with Mongoose

OR

- MySQL

Authentication

Use:

- JWT Authentication
- bcryptjs password hashing
- Protected routes

PROJECT STRUCTURE

The project folder structure must be clean and professional.

```
project-name/
|
|— backend-project/
|   |— config/
|   |— controllers/
|   |— middleware/
|   |— models/
|   |— routes/
|   |— server.js
|   |— .env
|
|— frontend-project/
|   |— src/
|   |   |— api/
|   |   |— assets/
|   |   |— components/
|   |   |— context/
|   |   |— layouts/
|   |   |— pages/
|   |   |— routes/
|   |   |— App.jsx
|   |   |— main.jsx
|   |— package.json
```

DATABASE DESIGN SECTION

Create database tables/collections based on the following entities:

[ADD DATABASE ENTITIES HERE]

For each entity include:

- Primary Key
 - Foreign Keys
 - Relationships
 - Validation rules
-

ERD REQUIREMENTS

Create a complete ERD showing:

- Entities
 - Attributes
 - Primary Keys
 - Foreign Keys
 - Relationships
 - Cardinalities
-

APPLICATION REQUIREMENTS

Build a complete management system with the following modules:

- Authentication Module
 - Dashboard Module
 - Main Entity Management
 - Reports Module
 - Profile Module
-

AUTHENTICATION FEATURES

Implement:

User Registration

- Username
- Email
- Password
- Confirm Password

Login System

- Secure login
- JWT token authentication
- Logout functionality

Security

- Password encryption using bcryptjs

- Duplicate account prevention
 - Protected routes
 - Authentication middleware
-

DASHBOARD FEATURES

Create a professional dashboard showing:

- Total records
- Statistics cards
- Recent activities
- Charts/graphs
- Quick actions

Dashboard must look modern and responsive.

CRUD OPERATIONS

Implement complete CRUD functionality.

Modules Requiring CRUD

[ADD CRUD MODULES HERE]

Example:

- Student
- Employee
- Product
- Parking Record

Implement:

- Create
 - Read
 - Update
 - Delete
-

FORM FEATURES

All forms must include:

- Validation
- Error handling
- Success messages
- Loading states
- Required fields
- Responsive design

Use:

- Controlled Components

OR

- React Hook Form
-

UI/UX REQUIREMENTS

Design must be:

- Professional
- Modern
- Clean
- Responsive
- Attractive

Use:

- Tailwind CSS
 - Flexbox
 - CSS Grid
 - Hover effects
 - Focus states
 - Smooth transitions
 - Shadows
 - Rounded corners
 - Icons
-

RESPONSIVENESS

The application must work perfectly on:

- Mobile phones
- Tablets
- Laptops
- Desktop screens

Use Tailwind responsive breakpoints properly.

NAVIGATION

Create a professional sidebar/navbar with routes such as:

[ADD NAVIGATION LINKS HERE]

Example:

- Dashboard
- Students
- Employees
- Reports
- Settings
- Logout

Implement:

- React Router DOM
 - Active links
 - Protected navigation
-

API INTEGRATION

Frontend must communicate with backend using Axios.

Create:

src/api/

Inside it create API files such as:

[ADD API FILES HERE]

Example:

- authApi.js
- studentApi.js
- paymentApi.js
- reportApi.js

Configure:

- Axios instance
 - Base URL
 - Error handling
-

BACKEND REQUIREMENTS

Create RESTful APIs using Express.js.

Implement:

POST

Create data

GET

Retrieve data

PUT

Update data

DELETE

Delete data

Use:

- Express Router

- Controllers
 - Middleware
 - Environment variables
-

DATABASE REQUIREMENTS

Database must include:

- Proper collections/tables
 - Primary keys
 - Foreign keys/references
 - Relationships between entities
 - Validation rules
-

REPORT FEATURES

Generate professional reports including:

[ADD REPORT TYPES HERE]

Example:

- Daily Reports
- Monthly Reports
- Payment Reports
- Attendance Reports

Optional:

- Export PDF
 - Print reports
 - Export Excel
-

SEARCH AND FILTER FEATURES

Implement:

- Search bar
- Filter dropdowns
- Sorting

- Pagination
-

ERROR HANDLING

Implement proper:

- Backend error handling
 - Frontend error handling
 - Try/catch blocks
 - Validation messages
 - API error responses
-

LOADING FEATURES

Add:

- Loading spinners
 - Disabled submit buttons while loading
-

CODE QUALITY

Code must be:

- Clean
- Reusable
- Modular
- Scalable
- Well structured

Use:

- Reusable components
 - Custom hooks where needed
-

OPTIONAL ADVANCED FEATURES

If possible include:

- Dark mode

- Notifications
 - Charts and analytics
 - PDF export
 - Image upload
 - Activity logs
-

FINAL OUTPUT REQUIREMENTS

The final application must include:

- Professional UI/UX
- Fully working frontend
- Fully working backend
- Database integration
- Authentication system
- CRUD operations
- Reports generation
- Responsive design
- Clean folder structure
- Professional code architecture

The application should look like a real production-level MERN application suitable for National Practical Exams and professional portfolio presentation.

NOW CONTINUE WITH THIS PROJECT

[PASTE YOUR PROJECT / NATIONAL EXAM QUESTION HERE]

Then continue generating:

- ERD
- Database schema
- Backend
- Frontend
- APIs
- Authentication
- UI/UX
- Reports
- CRUD operations
- Axios integration
- Tailwind components
- Responsive pages

- Professional folder structure